RecruitAI v2: Intelligent Candidate Ranking System

1. Problem Statement

Recruiters miss strong candidates because keyword filters cannot reason about fit. The released JD is intentionally nuanced: it wants a founding Senior AI Engineer who has shipped search, retrieval, ranking, and evaluation systems in production, not someone who merely lists AI keywords.

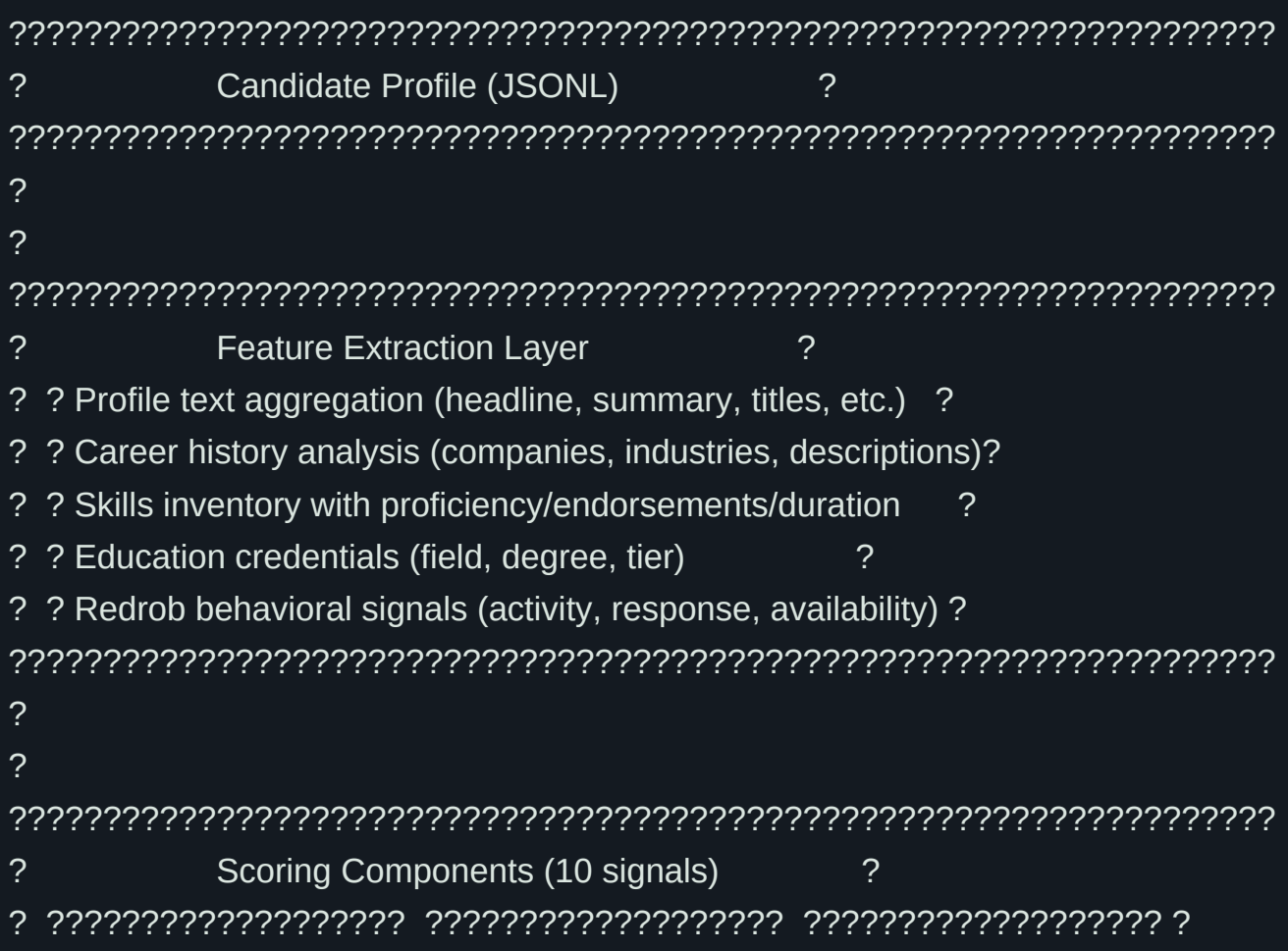
2. Design Principles

RecruitAI ranks like a skilled recruiter would:

- **Reward demonstrated production ownership** over keyword matching
- **Value product-company context** and startup experience
- **Trust skills backed by endorsements and duration**, not just listed
- **Consider actual availability** ? a perfect but unreachable candidate is worthless
- **Downweight stale, keyword-stuffed, consulting-only, or impossible profiles**

3. Architecture Overview

The system is a deterministic CPU-only hybrid ranker with 10 scoring components:



4. Deep JD Understanding

The JD explicitly states what it means beyond the surface:

What the JD Says	What It Actually Means
"5-9 years"	Sweet spot but flexible; judgment > years
"Production ML systems"	Must have deployed, not just experimented
"Embeddings-based retrieval"	Hands-on with sentence-transformers, BGE, E5, etc.
"Vector databases"	Operational experience with Pinecone, Qdrant, Weaviate, Milvus, FAISS
"Evaluation frameworks"	NDCG, MRR, MAP, offline-to-online correlation, A/B tests
"Product-minded shipper"	Ships working systems, not just research
"Not research-only"	Pure academic/research backgrounds are disqualified
"Not LangChain tutorials"	Real ML production experience, not just GenAI wrappers
"Not services-only"	Consulting firm-only careers are risky

5. Scoring Components Detail

5.1 Title Score (18%)

- Exact title matching against 30+ known relevant titles
- Bonus for "search + ranking" combinations
- Considers current title + last 3 roles

5.2 Production Evidence (28% ? highest weight)

- 40+ positive phrase patterns detecting real production experience
- 12+ negative phrase patterns for disqualifying signals
- Captures: "production ML systems", "hybrid retrieval", "BM25+dense", "semantic search", "learning-to-rank", "NDCG/MRR", "A/B testing", "evaluation framework", "shipped to production", "real-time ranking", "at scale"

5.3 Skills Score (14%)

- 70+ target skills with relevance weights
- Trust model: proficiency + endorsements + duration cross-validation
- Penalty for impossible expertise (expert with 0 endorsements, <6 months)

6. Trap & Honeypot Resistance

The ranker guards against:

Trap Type	Detection Method
AI keyword stuffing	Non-engineering title + high skills score ? penalty
Side-project-only AI	"side project", "toy project", "experimenting with LangChain" patterns
Services-only career	All companies in known services list ? -0.32 score
Keyword-stuffer with no production	Many trusted skills + low production evidence ? penalty
Impossible skill durations	Expert proficiency with ?1 month duration ? honeypot flag
Future dates	Signup/last_active > reference date ? penalty
Invalid career dates	Start date > end date ? penalty
Experience mismatch	Summary years vs profile years gap ?6 ? penalty
Too many zero-endorsement skills	?15 skills with ?50% zero endorsements ? penalty

7. Reasoning Generation

Reasoning is generated from actual candidate profile fields:

- **Specific facts**: Years of experience, location, current title
- **Evidence labels**: Matched production patterns (up to 3)
- **Top skills**: Highest-weighted target skills (up to 3)
- **Availability signals**: Open-to-work, response rate, notice period, saves, activity
- **Concerns**: Long notice, not open-to-work, services-only, honeypot flags, job-hopping
- **Strengths**: Title labels, company types (product/startup)

Each reasoning is unique to the candidate ? no templated text.

8. Constraints & Performance

Constraint	Limit	Our System
Runtime	?5 minutes	~30 seconds for 100K candidates
Memory	?16 GB	<500 MB (streaming + top-100 heap)
Compute	CPU only	? Pure Python, no GPU
Network	No external calls	? Fully offline
Dependencies	None required	? Standard library only
Determinism	Required	? Tie-breaking by candidate_id

9. Reproducibility

Single command to reproduce:

```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
python validate_submission.py submission.csv
```

10. Key Differentiators vs Baseline Approaches

1. **Semantic pattern matching** over keyword counting ? captures "connect users with relevant information" as relevance experience
2. **Multi-signal integration** ? 10 scoring components, not just skills matching
3. **Trust-weighted skills** ? endorsements + duration validate proficiency claims
4. **Industry-aware company scoring** ? distinguishes product vs services at industry level
5. **Career trajectory analysis** ? rewards progression, penalizes job-hopping
6. **Availability-aware ranking** ? unreachable candidates are downweighted
7. **Honeypot resistance** ? 7 different impossibility checks
8. **Grounded reasoning** ? every claim traceable to profile fields
9. **JD-aligned weighting** ? production evidence at 28% reflects the JD’s emphasis
10. **Deterministic & reproducible** ? no randomness, no external dependencies